ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
KHOA CÔNG NGHỆ THÔNG TIN

———————o0o———————



BÁO CÁO ĐỒ ÁN

Xv6 Utilities & System calls

Môn học: Hệ điều hành

Giảng viên: Phan Quốc Kỳ
Lê Hà Minh
Lê Giang Thanh

Lớp: 24_1

Nhóm: 14

Thành viên	**MSSV**
Đỗ Đức Thịnh	24120509
Bùi Ngọc Hiệp	24120309
Lê Hữu Vinh Hiển	24120179

Thành phố Hồ Chí Minh, 2026

Mục lục

1 Tổng quan **3**

1.1 Mục tiêu 3

1.2 Phân công công việc 3

2 Xv6 Utilities **4**

2.1 Memdump 4

2.1.1 Người thực hiện 4

2.1.2 Hướng giải 4

2.1.3 Luồng hoạt động 5

2.1.4 Kiểm thử và hạn chế 6

2.2 Find + Exec 7

2.2.1 Find 7

2.2.2 Exec 9

2.3 Không in dấu $ khi xử lý lệnh từ file 13

2.3.1 Người thực hiện 13

2.3.2 Hướng giải 13

2.3.3 Các file và hàm đã chỉnh sửa 13

2.3.4 Luồng hoạt động 14

2.3.5 Kiểm thử 14

2.4 Tab completion 14

2.4.1 Người thực hiện 14

2.4.2 Hướng giải 14

2.4.3 Các file và hàm đã chỉnh sửa 15

2.4.4 Luồng hoạt động 15

2.4.5 Kiểm thử 15

2.5 Lịch sử lệnh 16

2.5.1 Người thực hiện 16

2.5.2 Hướng giải 16

2.5.3 Các file và hàm đã chỉnh sửa 16
2.5.4 Luồng hoạt động 17
2.5.5 Kiểm thử 17

3 System calls **18**
3.1 Sandbox masking 18
3.1.1 Người thực hiện 18
3.1.2 Hướng giải 18
3.1.3 Các file và hàm đã chỉnh sửa 18
3.1.4 Luồng hoạt động 19
3.1.5 Kiểm thử 19
3.2 Sandbox with allowed pathnames 20
3.2.1 Người thực hiện 20
3.2.2 Hướng giải 20
3.2.3 Các file và hàm đã chỉnh sửa 20
3.2.4 Luồng hoạt động 21
3.2.5 Kiểm thử 21
3.3 Attack xv6 22
3.3.1 Người thực hiện 22
3.3.2 Mục tiêu và nguyên nhân 22
3.3.3 Hướng giải 22
3.3.4 Các file và hàm đã chỉnh sửa 23
3.3.5 Luồng hoạt động 23
3.3.6 Kiểm thử và hạn chế 23
3.4 Bonus 24
3.4.1 Người thực hiện 24
3.4.2 Mô tả lỗi 25
3.4.3 Proof of concept 25
3.4.4 Cách sửa 26
3.4.5 Các file và hàm đã chỉnh sửa 26
3.4.6 Kiểm thử sau khi sửa 26

4 Chấm điểm **28**

Chương 1

Tổng quan

1.1 Mục tiêu

Đồ án này được thực hiện nhằm tìm hiểu và mở rộng hệ điều hành xv6 thông qua các yêu cầu lập trình hệ thống. Quá trình thực hiện giúp nhóm củng cố kiến thức về cơ chế quản lý tiến trình, thao tác trên hệ thống tệp, xử lý lệnh trong shell và tương tác giữa chương trình người dùng với kernel.

1.2 Phân công công việc

MSSV	**Thành viên**	**Hạng mục phụ trách**
24120509	Đỗ Đức Thịnh	Memdump; Find + Exec
24120309	Bùi Ngọc Hiệp	Cải tiến Shell; Sandbox a Command
24120179	Lê Hữu Vinh Hiển	Attack xv6; Bonus

Bảng 1.1: Phân công công việc giữa các thành viên

Bảng phân công thể hiện trách nhiệm chính của từng thành viên. Trong quá trình thực hiện, các thành viên có thể hỗ trợ lẫn nhau khi xử lý lỗi, kiểm thử và tích hợp mã nguồn. Nội dung đóng góp cụ thể của từng thành viên trong mỗi hạng mục sẽ được trình bày tại chương tương ứng.

Chương 2

Xv6 Utilities

2.1 Memdump

2.1.1 Người thực hiện

Thành viên	Đỗ Đức Thịnh
MSSV	24120509
Nội dung phụ trách	Cài đặt hàm `memdump()` trong `user/memdump.c`: duyệt chuỗi định dạng, đọc dữ liệu từ vùng nhớ theo từng kiểu (`i`, `p`, `h`, `c`, `s`, `S`) và in ra màn hình.

2.1.2 Hướng giải

Hàm `memdump()` duyệt chuỗi định dạng `fmt` từ trái sang phải. Mỗi ký tự của `fmt` quy định kiểu dữ liệu cần lấy ra từ vùng nhớ bắt đầu tại `data` và cách in giá trị đó. Sáu định dạng đã được cài đặt:

- `i`: số nguyên 32 bit, in thập phân (4 byte).
- `p`: giá trị 64 bit, in thập lục phân (8 byte).
- `h`: số nguyên 16 bit, in thập phân (2 byte).
- `c`: một ký tự (1 byte).
- `s`: con trỏ 8 byte trỏ tới chuỗi, in chuỗi được trỏ tới.
- `S`: chuỗi nằm trực tiếp tại vị trí hiện tại của `data`, tiêu thụ toàn bộ số byte còn lại.

Chương trình dùng biến `cntByte` (khởi tạo bằng `len`) để theo dõi số byte chưa đọc. Trước mỗi lần đọc, `cntByte` được so với kích thước kiểu dữ liệu cần đọc. Nếu không đủ, chương trình in thông báo `memdump: not enough data for '<ký tự>'` và thoát với mã lỗi 1. Nếu đủ, giá trị được lấy bằng `memcpy()` vào biến cục bộ đúng kiểu, in ra, rồi `data` tiến lên và `cntByte` giảm đúng bằng số byte đã dùng.

2.1.2.1 Các file và hàm đã chỉnh sửa

File user/memdump.c

Hàm `memdump(char *fmt, char *data, int len)` xử lý dữ liệu theo chuỗi định dạng `fmt`. Chương trình duyệt từng ký tự định dạng, xác định số byte cần đọc, kiểm tra độ dài dữ liệu còn lại và thực hiện đọc, in kết quả tương ứng. Biến `cntByte` theo dõi số byte còn lại. Sau mỗi lần xử lý, chương trình cập nhật con trỏ `data` và giảm `cntByte` theo số byte đã sử dụng. Với các định dạng có độ dài cố định, chương trình kiểm tra dữ liệu trước khi đọc; nếu không đủ byte, chương trình thông báo lỗi và kết thúc.

2.1.3 Luồng hoạt động

1. Hàm nhận `fmt`, `data`, `len`; khởi tạo `cntByte = len`.

2. Lấy ký tự định dạng hiện tại `fmt[i]`; nếu là `'\0'` thì kết thúc.

3. Xác định số byte cần đọc theo ký tự định dạng: 4 byte với `i`, 8 byte với `p`, 2 byte với `h`, 1 byte với `c`, 8 byte với `s`, và toàn bộ số byte còn lại với `S`.

4. Với các định dạng `i`, `p`, `h`, `c`, `s`, nếu `cntByte` nhỏ hơn số byte cần đọc, chương trình in thông báo lỗi tương ứng và gọi `exit(1)`. Riêng định dạng `S`, chương trình duyệt tối đa `cntByte` byte, dừng khi gặp byte null hoặc hết dữ liệu hợp lệ.

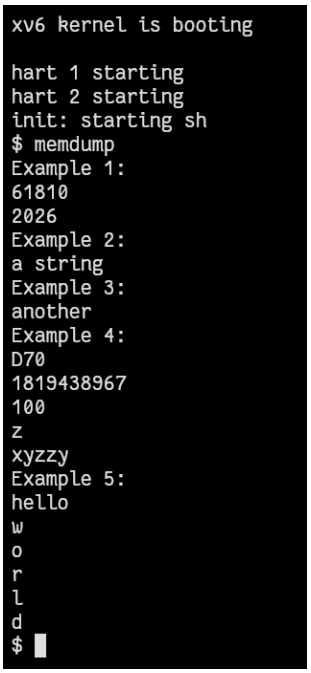
5. Ngược lại: dùng `memcpy()` lấy giá trị vào biến cục bộ, in theo định dạng tương ứng.

6. Tăng `data`, giảm `cntByte` theo số byte đã dùng, tăng `i` và quay lại bước 2.

2.1.4 Kiểm thử và hạn chế

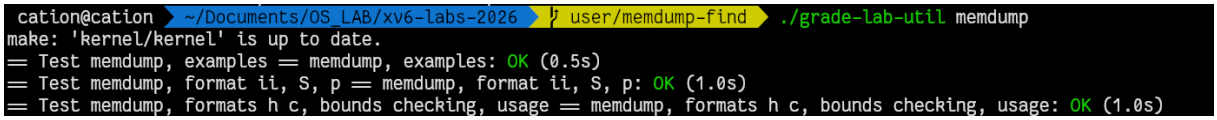
Kiểm thử

Trong xv6, ta gọi memdump không tham số để chạy dữ liệu mẫu:



Hình 2.1: Kết quả khi chạy memdump trong xv6 với dữ liệu mẫu

Trong terminal, ta thực hiện lệnh `./grade-lab-util memdump` để kiểm thử memdump.



Hình 2.2: Kết quả khi chạy kiểm thử memdump

Hạn chế

- Hiện đề chưa có yêu cầu xử lý cho kí tự format không hợp lệ nên chưa có nhánh `default` trong `switch`, vì vậy ký tự định dạng không hợp lệ bị bỏ qua mà không báo lỗi.

2.2 Find + Exec

2.2.1 Find

2.2.1.1 Người thực hiện

Thành viên	Đỗ Đức Thịnh
MSSV	24120509
Nội dung phụ trách	Cài đặt cơ chế duyệt đệ quy cây thư mục và tìm kiếm tệp theo tên trong `user/find.c`.

2.2.1.2 Hướng giải

Phần chức năng `find` chịu trách nhiệm duyệt cây thư mục bằng đệ quy bắt đầu từ đường dẫn gốc được chỉ định. Thuật toán hoạt động như sau:

- Mở thư mục bằng hàm `open()` với quyền đọc và kiểm tra tính hợp lệ bằng `fstat()` (đảm bảo tệp có kiểu là `T_DIR`).
- Duyệt tuần tự qua các mục trong thư mục bằng hàm `read()` trả về cấu trúc `struct dirent`.
- Bỏ qua các mục không hợp lệ (`de.inum == 0`) hoặc các thư mục điều hướng đặc biệt (`.` và `..`).
- Xây dựng đường dẫn đầy đủ của tệp/thư mục con và gọi `stat()` để lấy thông tin chi tiết.
- So sánh tên tệp với tên cần tìm (`name`). Nếu khớp và không có lệnh thực thi kèm theo, in đường dẫn tệp ra màn hình.
- Nếu mục hiện tại là thư mục con, đệ quy gọi lại hàm `find()` để tiếp tục quét bên trong.

2.2.1.3 Các file và hàm đã chỉnh sửa

File `user/find.c` (Phần tìm kiếm)

Hàm `find()` thực hiện tìm kiếm đệ quy trong cây thư mục, bắt đầu từ đường dẫn được cung cấp. Chương trình mở đường dẫn bằng `open()`, sử

dụng `fstat()` để kiểm tra thuộc tính và chỉ tiếp tục duyệt khi đối tượng là thư mục hợp lệ. Nếu thao tác thất bại, hàm đóng tài nguyên khi cần và kết thúc xử lý đường dẫn hiện tại.

Trong quá trình duyệt, chương trình sử dụng `read()` để đọc lần lượt các mục thư mục thông qua cấu trúc `struct dirent`. Những mục không hợp lệ và hai mục đặc biệt `.`, `..` được bỏ qua để tránh xử lý không cần thiết và đệ quy ngược lên thư mục cha.

Với mỗi mục hợp lệ, chương trình nối tên mục vào đường dẫn hiện tại bằng bộ đệm `buf`. Trước khi nối, hàm kiểm tra độ dài đường dẫn để tránh vượt quá kích thước bộ đệm. Sau đó, `stat()` được sử dụng để lấy thông tin của mục vừa tạo đường dẫn.

```
memmove(p, de.name, DIRSIZ);
p[DIRSIZ] = '\0';

if (stat(buf, &st) < 0)
    continue;

if (strcmp(de.name, name) == 0) {
    if (cmd == 0)
        printf("%s\n", buf);
    else {
        // Xu ly thuc thi lenh -exec
    }
}

if (st.type == T_DIR)
    find(buf, name, cmd);
```

Đoạn code trên thể hiện việc xây dựng đường dẫn, kiểm tra thông tin mục con, so khớp tên cần tìm và tiếp tục duyệt các thư mục con. Khi tên mục trùng với tên cần tìm, chương trình in đường dẫn nếu chạy ở chế độ tìm kiếm thông thường; nếu có yêu cầu `-exec`, chương trình chuyển sang nhánh thực thi lệnh tương ứng.

Sau khi duyệt hết các mục, hàm đóng mô tả tệp bằng `close(fd)` để giải phóng tài nguyên.

File `Makefile`

Bổ sung chương trình `_find` vào danh sách `UPROGS` để hệ thống biên dịch và đưa chương trình tìm kiếm vào ảnh hệ thống tệp của xv6 trong

quá trình build.

2.2.1.4 Luồng hoạt động của phần Find

1. Nhận đường dẫn `path`, tên tệp `name` và con trỏ lệnh `cmd`.
2. Kiểm tra xem `path` có mở được không và có phải là thư mục hợp lệ hay không.
3. Tạo bộ đệm chứa đường dẫn dài đầy đủ cho từng tệp con bên trong thư mục.
4. Đọc từng entry qua `struct dirent`, lọc bỏ `.` và `..`
5. So khớp tên tệp. Nếu trùng khớp và không yêu cầu `-exec` (`cmd == 0`), in đường dẫn `buf` ra màn hình.
6. Nếu gặp thư mục con, tiếp tục gọi đệ quy `find()` cho đến khi quét toàn bộ cây thư mục.

2.2.2 Exec

2.2.2.1 Người thực hiện

Thành viên	Đỗ Đức Thịnh
MSSV	24120509
Nội dung phụ trách	Cài đặt tính năng thực thi lệnh `-exec` tích hợp trong `user/find.c`.

2.2.2.2 Hướng giải

Phần `exec` mở rộng tiện ích `find` cho phép chạy một chương trình/lệnh hệ thống bất kỳ đối với mỗi tệp tìm thấy:

- Phân tích cú pháp dòng lệnh trong hàm `main()` để nhận diện tùy chọn `-exec` và trích xuất danh sách các đối số lệnh đi kèm.

- Khi một tệp khớp với điều kiện tìm kiếm và có yêu cầu `-exec` (`cmd != 0`), chương trình xây dựng mảng đối số `args`, sao chép các tham số lệnh và gắn đường dẫn tệp tìm được vào vị trí cuối cùng của mảng đối số.

- Sử dụng hệ thống gọi tiến trình `fork()` để tạo tiến trình con. Trong tiến trình con, gọi `exec()` để thực thi lệnh; tiến trình cha gọi `wait()` để chờ tiến trình con thực thi xong trước khi tiếp tục quét.

2.2.2.3 Các file và hàm đã chỉnh sửa

File `user/find.c` (Phần thực thi lệnh)

Khi tìm thấy mục có tên khớp với yêu cầu và chế độ `-exec` được bật, chương trình xây dựng danh sách đối số để truyền cho hàm `exec()`. Danh sách bao gồm tên chương trình, các tham số do người dùng cung cấp và đường dẫn của mục vừa tìm được.

```
char *args[MAXARG];
int i = 0;

while (cmd[i] && i < MAXARG - 2) {
    args[i] = cmd[i];
    i++;
}

args[i++] = buf;
args[i] = 0;
```

Mảng `args` lưu các đối số theo định dạng yêu cầu của `exec()`. Vòng lặp sao chép các tham số từ `cmd`, giới hạn số lượng để dành chỗ cho đường dẫn tìm được và phần tử kết thúc mảng. Đường dẫn `buf` được bổ sung vào cuối danh sách, sau đó phần tử `0` đánh dấu kết thúc.

Sau khi chuẩn bị đối số, chương trình sử dụng `fork()` để tạo tiến trình con thực thi lệnh. Tiến trình cha gọi `wait(0)` để chờ tiến trình con hoàn thành trước khi tiếp tục duyệt các mục khác.

```
int pid = fork();

if (pid == 0) {
    exec(args[0], args);
    fprintf(2, "find: exec failed\n");
```

```
6       exit(1);
7   }
8
9   if (pid > 0)
10      wait(0);
```

Trong tiến trình con, `exec()` thay thế chương trình hiện tại bằng chương trình cần thực thi. Nếu thực thi thất bại, chương trình in thông báo lỗi và kết thúc tiến trình con. Tiến trình cha chờ tiến trình con kết thúc để bảo đảm các lệnh được xử lý tuần tự.

Hàm `find_exec()`

Hàm `find_exec()` nhận đường dẫn bắt đầu tìm kiếm, tên cần tìm và danh sách đối số của lệnh. Hàm chuyển các tham số cho `find()` để sử dụng lại cơ chế duyệt đệ quy, so khớp tên và xử lý thực thi lệnh đã cài đặt.

Hàm `main()`

Hàm `main()` kiểm tra tham số dòng lệnh và lựa chọn chế độ hoạt động. Nếu có ba tham số, chương trình gọi `find()` để tìm kiếm thông thường. Nếu có thêm tham số, chương trình kiểm tra từ khóa `-exec` và yêu cầu có ít nhất một đối số lệnh.

```
1   if (argc == 3) {
2       find(argv[1], argv[2], 0);
3   } else if (argc >= 5 && strcmp(argv[3], "-exec") == 0) {
4       find_exec(argv[1], argv[2], &argv[4]);
5   } else {
6       fprintf(2, "Usage: find path filename [-exec cmd ...]\n");
7       exit(1);
8   }
```

Trong chế độ `-exec`, `&argv[4]` được truyền làm vị trí bắt đầu của danh sách đối số lệnh. Nếu cú pháp không hợp lệ, chương trình in hướng dẫn sử dụng và kết thúc với mã lỗi. Sau khi hoàn thành, chương trình gọi `exit(0)` để kết thúc bình thường.

2.2.2.4 Luồng hoạt động của phần Exec

1. Người dùng chạy chương trình với cú pháp bổ sung `-exec cmd ...`.
2. Hàm `main()` kiểm tra cú pháp, nếu hợp lệ sẽ gọi `find_exec()` chuyển

tiếp danh sách đối số lệnh (`&argv[4]`).

3. Khi hàm `find()` tìm thấy tệp trùng khớp với `name` và có cờ `cmd` khác rỗng:

   - Sao chép các tham số vào mảng `args`.
   - Nối đường dẫn tệp tìm được vào phần tử kế cuối và kết thúc mảng bằng con trỏ `0`.
   - Gọi `fork()` để tách tiến trình. Tiến trình con gọi `exec(args[0], args)`, tiến trình cha gọi `wait(0)` chờ đợi.

4. Tiến trình con thực thi chương trình hệ thống yêu cầu trên tệp vừa tìm được.

2.2.2.5 Kiểm thử và hạn chế

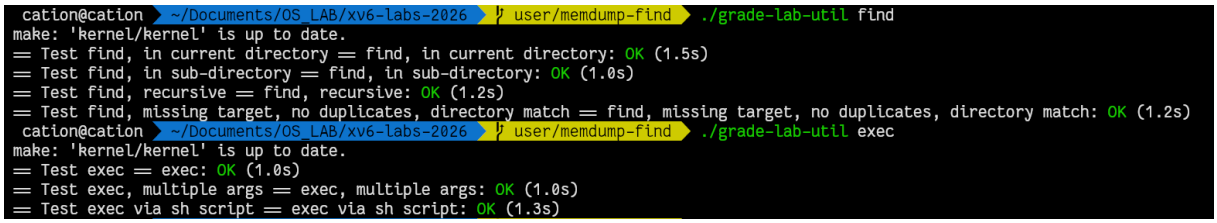
Kiểm thử

Ta chạy thử một vài test trong xv6 như sau:



Hình 2.3: Kết quả khi chạy find + exec trong xv6 với test trong tài liệu

Trên terminal, ta chạy `./grade-lab-util find` và `./grade-lab-util exec` để kiểm thử tính năng find và find + exec.



Hình 2.4: Kết quả khi chạy kiểm thử find + exec

2.3 Không in dấu $ khi xử lý lệnh từ file

2.3.1 Người thực hiện

Thành viên	Bùi Ngọc Hiệp
MSSV	24120309
Nội dung phụ trách	Phân biệt stdin từ console với stdin được chuyển hướng để shell không in prompt khi đọc lệnh từ file.

2.3.2 Hướng giải

Shell kiểm tra file descriptor 0 bằng `fstat()` và so sánh thông tin thiết bị với thiết bị `console`. Nếu stdin là console, shell in prompt và dùng chế độ nhập tương tác. Nếu stdin được chuyển hướng, shell tiếp tục đọc bằng `gets()` nhưng bỏ qua prompt. Nhờ vậy các lệnh trong script được thực thi tuần tự mà output không bị xen các dấu `$`.

2.3.3 Các file và hàm đã chỉnh sửa

- `user/sh.c` — `stdin_is_console()` kiểm tra stdin có cùng loại thiết bị, `dev` và `ino` với console hay không.
- `user/sh.c` — `getcmd()` chọn cách nhập dựa trên kết quả nhận diện:

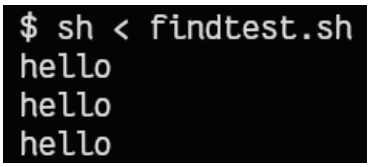
```
if (!interactive) {
  gets(buf, nbuf);
  return buf[0] == 0 ? -1 : 0;
}
write(2, "$ ", 2);
```

2.3.4 Luồng hoạt động

1. Nhận input từ `stdin`.
2. Kiểm tra thiết bị qua `fstat() / nhận diện console`.
3. Nếu là console: in `"$ "` → đọc dòng tương tác.
4. Nếu là file: đọc dòng không in prompt → `parsecmd` → thực thi.

2.3.5 Kiểm thử

Ta chạy thử `sh < findtest.sh` để thực hiện các lệnh trong file và kiểm tra xem có hiện các dấu `"$"` dư thừa hay không.



Hình 2.5: Kết quả khi chạy kiểm thử sh < findtest.sh

2.4 Tab completion

2.4.1 Người thực hiện

Thành viên	Bùi Ngọc Hiệp
MSSV	24120309
Nội dung phụ trách	Bổ sung hoàn thành tên lệnh và tên file bằng phím Tab.

2.4.2 Hướng giải

Khi người dùng nhấn Tab, shell xác định phần tiền tố của từ đang nhập và vị trí của từ đó trong dòng lệnh. Nếu là từ đầu tiên, shell duyệt thư mục gốc `/` và chỉ chọn các file có ELF magic hợp lệ làm chương trình. Nếu là đối số, shell tìm tên phù hợp trong thư mục hiện tại. Một kết quả thì shell điền phần còn lại; nhiều kết quả thì in danh sách trên dòng mới rồi vẽ lại prompt và nội dung đã nhập; không có kết quả thì giữ nguyên dòng.

Console ở chế độ tương tác chuyển Tab thành một sự kiện riêng cho shell. Cách này giúp shell xử lý phím ngay khi nhấn, thay vì phải đợi console gom đủ một dòng.

2.4.3 Các file và hàm đã chỉnh sửa

- `user/sh.c` — `complete_word()` xác định từ hiện tại, phân biệt lệnh/đối số và xử lý kết quả; `collect_completions()` duyệt thư mục và lọc tên; `is_executable()` kiểm tra ELF.
- `user/sh.c` — `getcmd()` gọi completion khi nhận sự kiện Tab:

```
if (c == KEY_TAB)
  complete_word(buf, &length, nbuf, interactive);
```

- `kernel/console.c` — trong chế độ raw của shell, chuyển ký tự Tab thành `KEY_TAB` và đánh thức tiến trình đang đọc.
- `kernel/sysproc.c`, `kernel/syscall.c`, `kernel/syscall.h`, `kernel/defs.h`, `user/user.h`, `user/usys.pl` — thêm syscall `consolemode` để bật/tắt chế độ nhập sự kiện.

2.4.4 Luồng hoạt động

1. `UART` nhận phím Tab → `consoleintr` tạo sự kiện `KEY_TAB`.
2. `consoleread` chuyển sự kiện cho shell.
3. `complete_word` duyệt thư mục phù hợp.
4. Điền duy nhất một kết quả hoặc in nhiều lựa chọn ra màn hình.

2.4.5 Kiểm thử

Trong xv6, ta gõ thử phần đầu của một lệnh/file nào đó rồi ấn phím `<TAB>` để hiện đầy đủ lệnh/file hoặc hiện tất cả các lệnh/file có cùng tiền tố với lệnh vừa nhập.



Hình 2.6: Kết quả khi chạy kiểm thử gr<TAB>

2.5 Lịch sử lệnh

2.5.1 Người thực hiện

Thành viên	Bùi Ngọc Hiệp
MSSV	24120309
Nội dung phụ trách	Lưu và duyệt lệnh gần đây; bổ sung `history`, `!!` và `!n`.

2.5.2 Hướng giải

Shell duy trì bộ đệm vòng gồm 10 lệnh. Mỗi lệnh mới được ghi vào vị trí `history_count % HISTORY_SIZE`, còn số thứ tự tổng được tăng liên tục để hỗ trợ cú pháp `!n`. Built-in `history` hiển thị các lệnh còn được lưu; `!!` tra lệnh gần nhất, `!n` tra lệnh theo số thứ tự. Trước khi chạy lại, shell in lệnh được mở rộng ra màn hình. Trên console, Up/Down duyệt các lệnh và cập nhật nội dung dòng nhập.

2.5.3 Các file và hàm đã chỉnh sửa

- `user/sh.c` — `history_add()` lưu lệnh vào bộ đệm vòng; `history_get()` tra cứu lệnh còn trong cửa sổ 10 lệnh; `history_reference()` phân tích `!!` và `!n`; `is_history_command()` nhận diện built-in `history`.
- `user/sh.c` — vòng lặp `main()` lưu lệnh, mở rộng tham chiếu lịch sử rồi thực thi hoặc in danh sách.
- `user/sh.c` — `getcmd()` xử lý Up/Down, xóa và vẽ lại dòng hiện tại.
- `kernel/console.c` — nhận chuỗi escape của phím mũi tên (`ESC [ A/B`) và chuyển thành `KEY_UP`/`KEY_DOWN`.

```
int slot = history_count % HISTORY_SIZE;
int i;
for (i = 0; i < sizeof(history[slot]) - 1 && line[i]; i++)
  history[slot][i] = line[i];
history[slot][i] = 0;
history_count++;
```

2.5.4 Luồng hoạt động

1. Gõ lệnh → lưu vào `history` → thực thi.
2. Lệnh `history` → in các lệnh đã lưu.
3. `!!`/`!n` hoặc phím Up/Down → tra cứu → hiển thị lệnh → thực thi khi nhấn Enter.

2.5.5 Kiểm thử

Trong xv6, ta chạy thử một vài lệnh, sau đó dùng các lệnh:

```
history
!!
!n
```

để in ra lịch sử, chạy lại lệnh cuối cùng và chạy lại lệnh thứ n

```
$ sh < findtest.sh
hello
hello
hello
$ echo a
a
$ history
1  sh < findtest.sh
2  echo a
3  history
$ !!
history
1  sh < findtest.sh
2  echo a
3  history
4  history
$ !2
echo a
a
```

Hình 2.7: Kết quả khi chạy kiểm thử

Chương 3

System calls

3.1 Sandbox masking

3.1.1 Người thực hiện

Thành viên	Bùi Ngọc Hiệp
MSSV	24120309
Nội dung phụ trách	Cài đặt syscall interpose, chặn syscall theo bitmask và kế thừa policy qua fork().

3.1.2 Hướng giải

Chương trình sandbox tạo process con, gọi interpose(mask, "), sau đó exec() chương trình cần giới hạn. Kernel lưu mask trong struct proc. Mỗi lần vào syscall(), kernel kiểm tra bit thứ num; nếu bit được đặt, syscall bị từ chối và trả -1 mà không gọi handler. kfork() sao chép mask để policy tiếp tục áp dụng cho các process con. Giá trị path "biểu thị không có ngoại lệ pathname.

3.1.3 Các file và hàm đã chỉnh sửa

- user/user.h, user/usys.pl — khai báo interpose() và sinh stub RISC-V thực hiện ecall.
- kernel/syscall.h, kernel/syscall.c — cấp số hiệu, đăng ký sys_interpose và đặt kiểm tra mask trước khi gọi handler.
- kernel/sysproc.c — sys_interpose() nhận mask bằng argint() và cài policy của process.

- kernel/proc.h, kernel/proc.c — thêm trường mask, khởi tạo khi cấp lại process và sao chép trong kfork().
- Makefile — thêm _sandbox và _sbtest vào UPROGS khi build syscall lab.

```
int denied = num < 32 && (((uint)p->mask & (1U << num)) != 0);
p->trapframe->a0 = denied ? -1 : syscalls[num]();
```

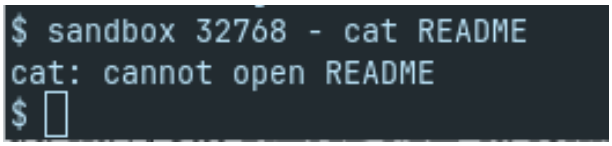
3.1.4 Luồng hoạt động

1. user/sandbox gọi fork → interpose(mask, '') → exec(command).
2. Thực thi ecall → syscall() kiểm tra mask.
3. Nếu hợp lệ gọi handler, ngược lại trả về -1.

3.1.5 Kiểm thử

Kiểm thử bằng cách chạy lệnh sau trong xv6:

`sandbox 32768 - cat README`



Hình 3.1: Kết quả khi chạy kiểm thử sandbox 32768 - cat README

Hoặc chạy `./grade-lab-syscall sandbox_mask` trên terminal



Hình 3.2: Kết quả khi chạy kiểm thử ./grade-lab-syscall sandbox_mask

3.2 Sandbox with allowed pathnames

3.2.1 Người thực hiện

Thành viên	Bùi Ngọc Hiệp
MSSV	24120309
Nội dung phụ trách	Lưu pathname ngoại lệ, kiểm tra open/exec theo pathname và kế thừa path qua fork().

3.2.2 Hướng giải

interpose(mask, path) lưu cả mask và pathname được phép trong process. Nếu open hoặc exec bị mask, kernel lấy chuỗi pathname từ tham số đầu tiên của syscall bằng fetchstr() rồi so sánh chính xác với path đã cấu hình. Path khớp thì syscall được chuyển tới handler; path không khớp hoặc không đọc được từ user space thì bị từ chối. kfork() sao chép cả path lẫn mask để tiến trình con chịu cùng policy.

3.2.3 Các file và hàm đã chỉnh sửa

- kernel/proc.h — thêm path[MAXPATH] và cờ interposed bên cạnh mask.
- kernel/sysproc.c — sys_interpose() nhận pathname bằng argstr() và lưu trong process; nếu policy đã cài thì không cho thay thế để tránh tự nới lỏng sandbox.
- kernel/proc.c — kfork() sao chép mask, path và trạng thái policy.
- kernel/syscall.c — với SYS_open/SYS_exec bị mask, lấy pathname từ a0; cho qua khi khớp path được phép. Path "không tạo ngoại lệ.

```
if (fetchstr(p->trapframe->a0, path, sizeof(path)) >= 0 &&
    strncmp(path, p->path, MAXPATH) == 0)
  denied = 0;
```

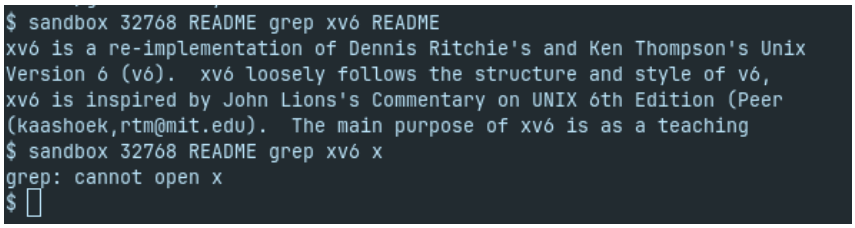
3.2.4 Luồng hoạt động

1. interpose(mask, path) lưu policy → fork kế thừa policy.
2. Gọi open/exec vào kernel → kiểm tra mask.
3. So khớp pathname nếu là open/exec: khớp thì thực thi, không khớp trả -1.

3.2.5 Kiểm thử

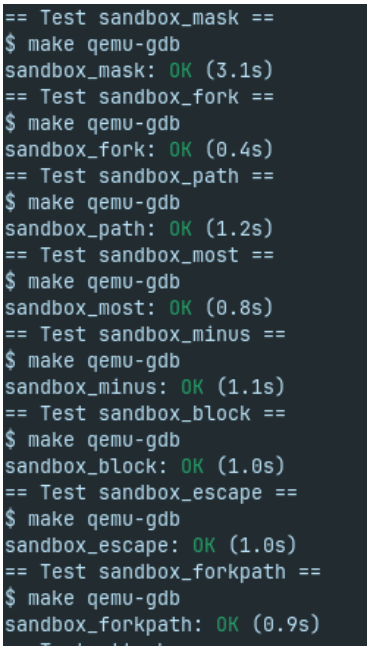
Kiểm thử bằng cách chạy các lệnh sau trong xv6:

```
sandbox 32768 README grep xv6 README
sandbox 32768 README grep xv6 x
```



Hình 3.3: Kết quả khi chạy kiểm thử sandbox

Hoặc chạy `make grade` trên terminal:



Hình 3.4: Kết quả khi chạy kiểm thử make grade

3.3 Attack xv6

3.3.1 Người thực hiện

Thành viên	Lê Hữu Vinh Hiển
MSSV	24120179
Nội dung phụ trách	Tìm secret đã được ghi vào bộ nhớ bằng attack.

3.3.2 Mục tiêu và nguyên nhân

Bài Attack xv6 yêu cầu tìm lại chuỗi bí mật mà chương trình `secret` đã ghi vào bộ nhớ trước khi kết thúc. Trong phiên bản xv6 dùng cho bài lab, một số lệnh khởi tạo và xóa nội dung trang nhớ đã bị tắt có chủ đích. Cụ thể, lệnh xóa trang trong hàm `uvmalloc()` của `kernel/vm.c` và hai lệnh ghi dữ liệu vào trang nhớ được giải phóng trong `kernel/kalloc.c` không được thực hiện khi biên dịch cấu hình này.

Do đó, sau khi một tiến trình kết thúc, trang nhớ được giải phóng có thể vẫn chứa dữ liệu cũ. Nếu tiến trình khác được cấp phát lại những trang này, dữ liệu trước đó có thể được đọc lại. Đây là nguyên nhân cho phép chương trình `attack` khai thác lỗi và lấy được secret của tiến trình đã kết thúc.

3.3.3 Hướng giải

Chương trình `secret` ghi chuỗi bí mật vào bộ nhớ cùng với tiền tố nhận diện `Here it is: `, sau đó kết thúc. Phần bộ nhớ được giải phóng có thể vẫn giữ lại chuỗi này do các thao tác xóa nội dung trang nhớ đã bị tắt trong bài lab.

Trong `user/attack.c`, chương trình sử dụng `sbrk()` để cấp phát thêm 32 trang nhớ, sau đó quét các vùng nhớ cần thiết để tìm tiền tố `Here it is: `. Khi phát hiện tiền tố, chương trình kiểm tra phần dữ liệu theo sau để xác định nội dung có đúng định dạng secret hay không.

Chương trình duyệt các ký tự chữ và số cho đến khi gặp ký tự kết thúc chuỗi (null terminator). Nếu chuỗi thỏa mãn điều kiện kiểm tra, chương

trình in secret ra một dòng riêng. Cách làm này không cố định độ dài secret ở 8 ký tự.

3.3.4 Các file và hàm đã chỉnh sửa

- `user/attack.c`: cài đặt các hàm quét vùng nhớ, kiểm tra ký tự chữ và số, tìm tiền tố nhận diện và lấy secret có độ dài thay đổi.
- `Makefile`: thêm chương trình `attack` vào danh sách chương trình người dùng để được biên dịch và đưa vào hệ thống tệp của xv6.

Phần khai thác không dựa vào việc sửa logic của kernel hoặc sửa chương trình `user/secret.c`. Việc tìm lại dữ liệu được thực hiện từ chương trình người dùng `attack`.

3.3.5 Luồng hoạt động

Quá trình thực hiện gồm các bước sau:

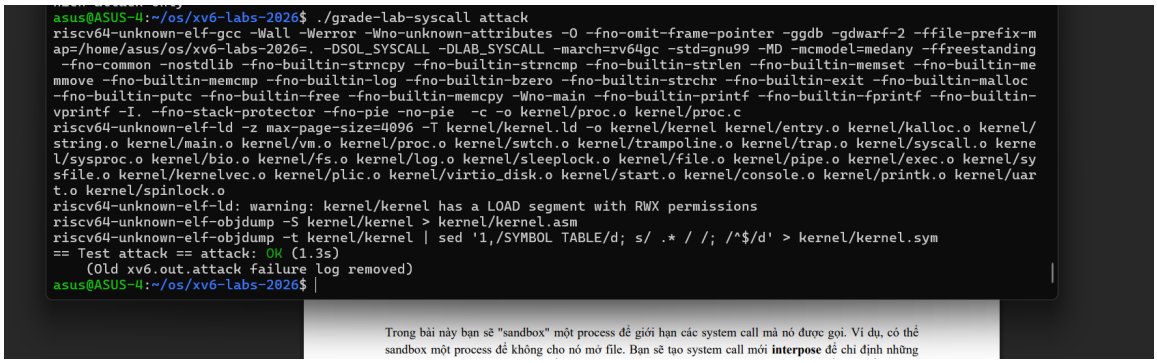
1. Chạy chương trình `secret` với một chuỗi đầu vào. Chương trình ghi secret vào bộ nhớ rồi kết thúc.
2. Các trang nhớ của tiến trình trước được giải phóng nhưng có thể vẫn giữ nội dung cũ.
3. Chạy chương trình `attack`. Chương trình mở rộng vùng heap bằng `sbrk()` và quét các vùng nhớ cần thiết để tìm chuỗi `Here it is: `.
4. Khi tìm thấy tiền tố, chương trình kiểm tra chuỗi theo sau, xác định độ dài dựa trên ký tự kết thúc chuỗi và kiểm tra các ký tự có đúng dạng chữ hoặc số hay không.
5. Nếu tìm được secret hợp lệ, chương trình in secret ra màn hình.

3.3.6 Kiểm thử và hạn chế

Kiểm thử riêng bài Attack bằng lệnh:

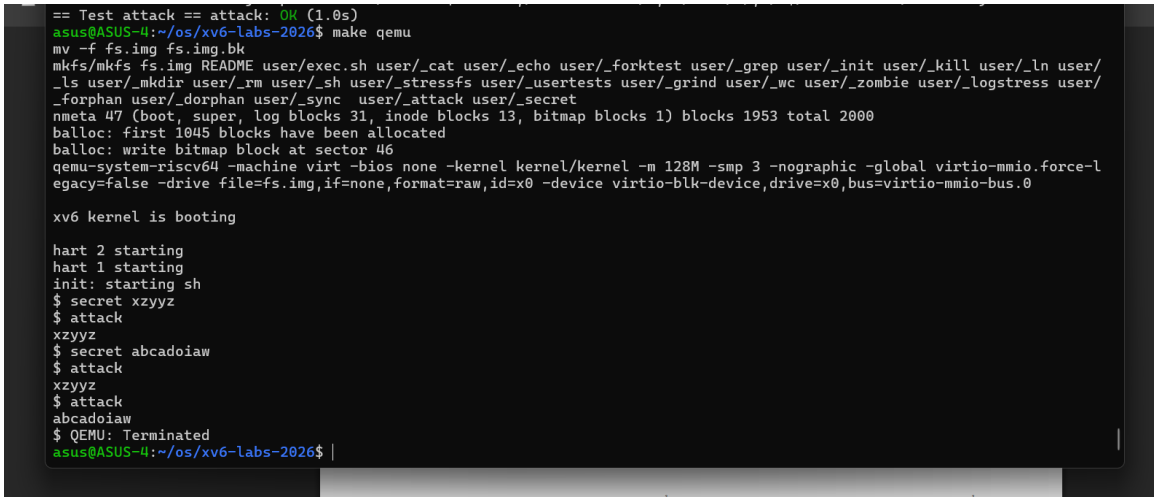
```
./grade-lab-syscall attack
```

Ảnh dưới đây ghi lại kết quả kiểm thử tự động ở lần chạy đã chụp:



Hình 3.5: Kết quả kiểm thử tự động bài Attack xv6.

Chương trình cũng được kiểm tra trực tiếp trong xv6 với secret `xyzzy`. Kết quả của lần chạy thành công được ghi nhận như sau:



Hình 3.6: Kiểm thử chương trình Attack trực tiếp trong xv6.

3.4 Bonus

3.4.1 Người thực hiện

Thành viên	Lê Hữu Vinh Hiển
MSSV	24120179
Nội dung phụ trách	Tìm bug trong xv6.

3.4.2 Mô tả lỗi

Lỗi được tìm thấy trong cơ chế xử lý system call `kill()` của xv6, cụ thể là hàm `kkill()` trong file `kernel/proc.c`.

Trong phiên bản ban đầu, hàm `kkill()` tìm tiến trình theo PID và đánh dấu tiến trình đó bằng cách đặt `p->killed = 1`. Hàm chưa loại trừ tiến trình hệ thống `init` khỏi thao tác này.

Trong xv6, tiến trình `init` là tiến trình quan trọng, có nhiệm vụ tiếp tục hoạt động và khởi chạy shell. Trong hàm `kexit()`, nếu tiến trình `init` kết thúc, kernel gọi `panic("init exiting")`. Vì vậy, nếu một chương trình người dùng gọi `kill(1)`, tiến trình `init` có thể bị đánh dấu kết thúc, dẫn đến kernel panic.

Lỗi này ảnh hưởng đến tính ổn định của toàn hệ thống. Một chương trình người dùng có thể gây crash kernel chỉ bằng một system call, không cần truy cập trực tiếp vào bộ nhớ kernel.

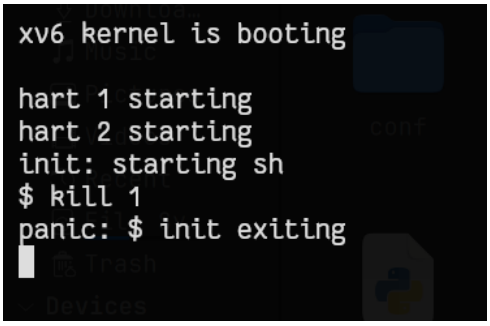
3.4.3 Proof of concept

Để tái hiện lỗi, chương trình `user/bonus.c` gọi system call `kill(1)` và in kết quả trả về. PID 1 là tiến trình `init` trong môi trường xv6 đang kiểm thử.

Trên phiên bản kernel chưa sửa, chạy chương trình `bonus` đã tái hiện lỗi với thông báo:

```
panic: init exiting
```

Kết quả cho thấy lời gọi từ chương trình người dùng có thể khiến kernel rơi vào trạng thái panic, làm gián đoạn hoạt động của hệ thống.



Hình 3.7: Kernel panic khi gọi kill(1) trước khi sửa lỗi.

3.4.4 Cách sửa

Phần sửa lỗi được thực hiện trong hàm `kkill()` của `kernel/proc.c`. Sau khi tìm được tiến trình phù hợp và giữ khóa của tiến trình đó, chương trình kiểm tra xem tiến trình có phải là `initproc` hay không.

Nếu đúng, hàm giải phóng khóa `p->lock` và trả về `-1`, không đặt cờ `p->killed`. Các yêu cầu `kill()` đối với những tiến trình khác vẫn tiếp tục được xử lý theo nhánh ban đầu.

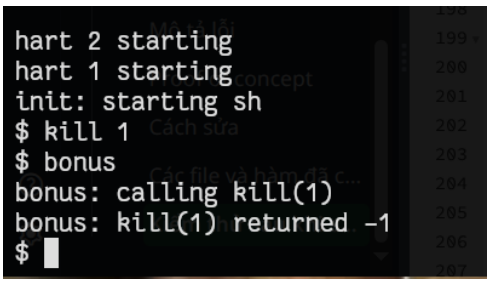
Việc kiểm tra này ngăn chương trình người dùng kết thúc tiến trình `init` thông qua system call `kill()`, qua đó tránh trường hợp kernel panic được mô tả ở trên.

3.4.5 Các file và hàm đã chỉnh sửa

- `kernel/proc.c`: bổ sung kiểm tra để từ chối yêu cầu kill tiến trình `initproc` trong hàm `kkill()`.
- `user/bonus.c`: chương trình kiểm thử gọi `kill(1)` và in giá trị trả về để quan sát kết quả.
- `Makefile`: thêm chương trình `bonus` vào danh sách chương trình người dùng được biên dịch.

3.4.6 Kiểm thử sau khi sửa

Kết quả kiểm thử trong môi trường xv6:



Hình 3.8: Kết quả kiểm thử Bonus sau khi sửa lỗi kill tiến trình init.

Sau khi sửa, system call `kill(1)` trả về `-1` và shell xv6 tiếp tục hoạt động. Trong lần kiểm thử này, thông báo `panic: init exiting` không

còn xuất hiện.

Trong terminal, ta chạy thử lệnh `make grade` để đảm bảo sửa đổi này không ảnh hưởng đến các phần khác:

```
$ make qemu-gdb
sandbox_mask: OK (3.0s)
== Test sandbox_fork ==
$ make qemu-gdb
sandbox_fork: OK (0.9s)
== Test sandbox_path ==
$ make qemu-gdb
sandbox_path: OK (1.1s)
== Test sandbox_most ==
$ make qemu-gdb
sandbox_most: OK (0.9s)
== Test sandbox_minus ==
$ make qemu-gdb
sandbox_minus: OK (1.0s)
== Test sandbox_block ==
$ make qemu-gdb
sandbox_block: OK (1.0s)
== Test sandbox_escape ==
$ make qemu-gdb
sandbox_escape: OK (1.0s)
== Test sandbox_forkpath ==
$ make qemu-gdb
sandbox_forkpath: OK (1.0s)
== Test attack ==
$ make qemu-gdb
attack: OK (1.1s)
```

Hình 3.9: Kết quả kiểm thử make grade

Chương 4

Chấm điểm

STT	Bài	Điểm	Tự chấm
1	memdump	1.5	1.5
2	find + exec	2.5	2.5
3	cải tiến shell	1.5	1.5
4	sandbox a command	2.5	2.5
5	attack xv6	2	2
6	bonus	2	2
	tổng	**12**	**12**